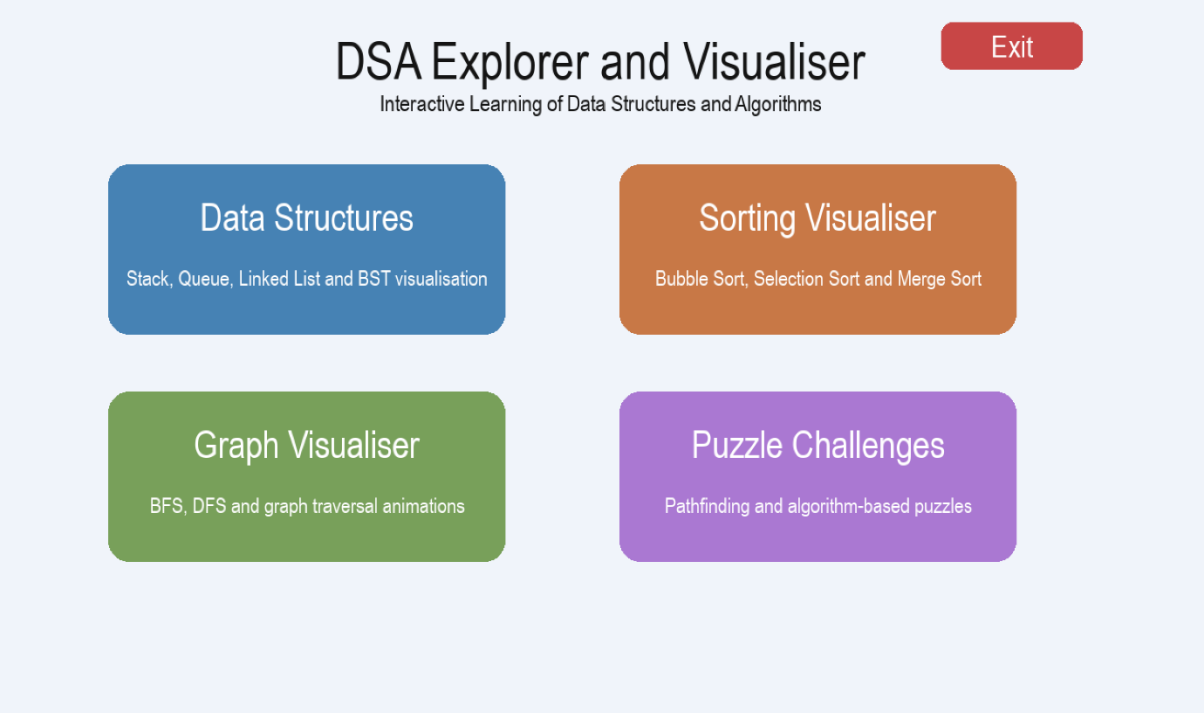
Data Structures and Algorithms Explorer and Visualiser Report

Introduction

This report covers the overall design, implementation, testing, and evaluation of the DSA Explorer and Visualiser application. The project was developed using Python and Pygame to create an interactive educational program that helps users understand data structures and algorithms visually.

This report explains and shows the design and testing process of the DSA Explorer and Visualiser application. This was developed utilising Python and Pygame to show computer science concepts that are within four main modules: Data Structures, Sort Visualiser, Graph Visualiser and the Puzzle Challenge, this can be seen in *Figure 1*.

Figure 1 – DSA Explorer and Visualiser UI



Data Structures Module

In the file data_structures.py, the module for Data Structures was created. There are graphical representations of Stack, Queue, Linked List, and Binary Search Tree structures. The Stack is an illustration of the Last In First Out principle through the Push and Pop operations, whereas the Queue is an illustration of the First In First Out principle through the Enqueue and Dequeue operations. These are shown in *Figure 2*.

The Linked List structure shows linked nodes with arrows in between each node; insertion, deletion, and reversal of nodes can be done. The Binary Search Tree structure shows linked nodes based on hierarchy; insertion of nodes is done based on the principles of the Binary Search Tree, and traversal can be done via inorder traversal. Various colours and labels were utilized to differentiate each structure, while textual information was provided at the bottom of the screen to indicate which operation was currently being done. This can be seen in *Figure 3*.

Figure 2 – Stack and Queue Features

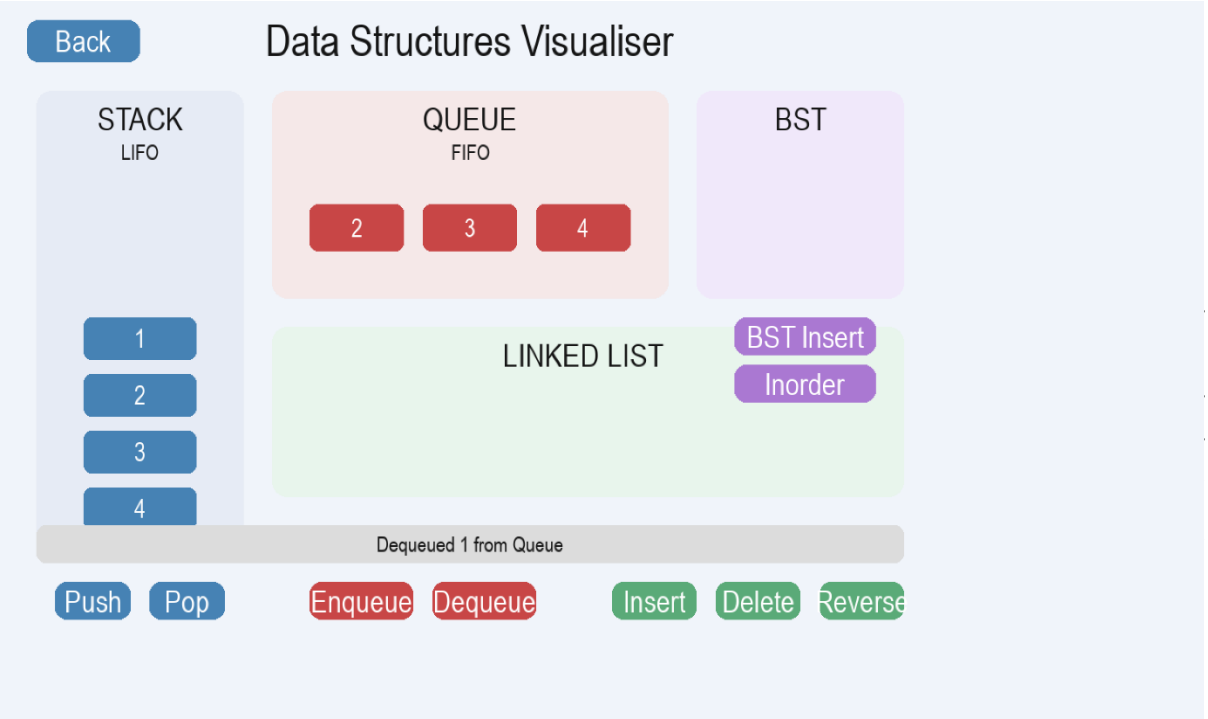
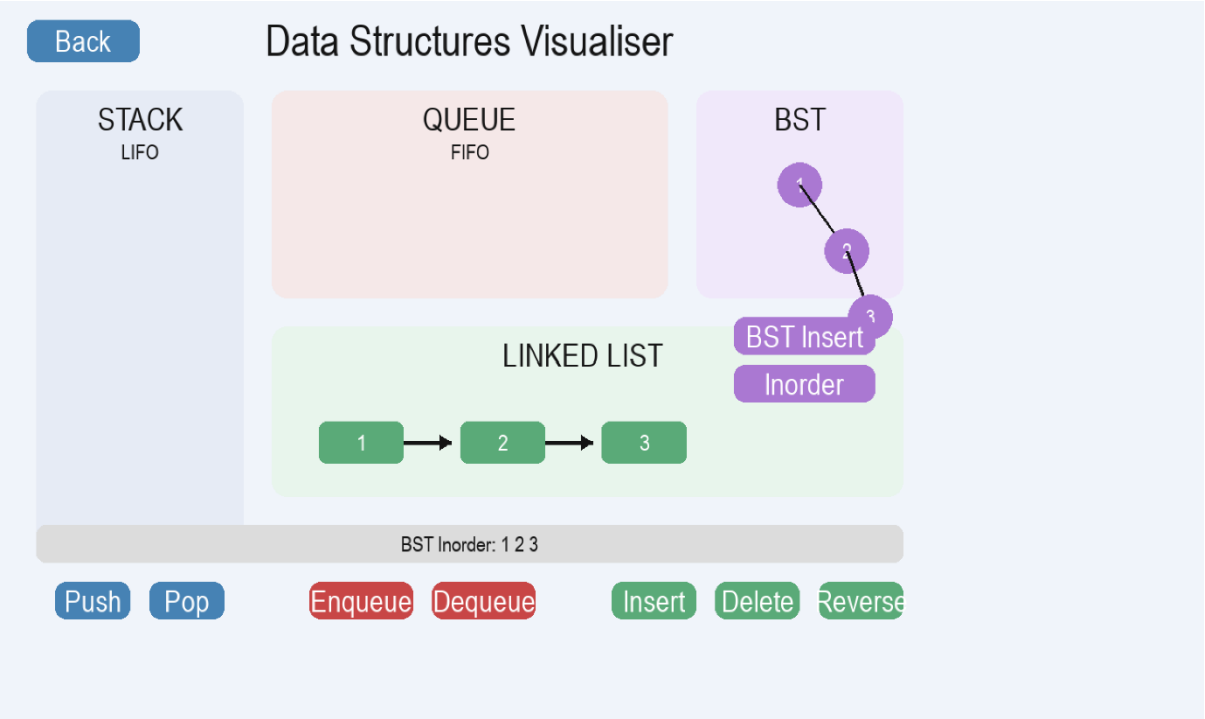


Figure 3 – BST and Linked List Features



Sorting Visualiser

Sorting Visualiser was created within the file called sorting.py. There are implementations of different sorting algorithms like Bubble Sort, Selection Sort and Merge Sort implemented in the code. While implementing Bubble and Selection sort algorithm, bars (depicting the numbers through heights) turn into different colours showing where the algorithm is now. Yellow depicts the number being compared with the number on its left side (red). Green shows the bars that were already sorted. (see *Figure 4*) With Selection sort, red shows the smallest number in comparison with the rest and yellow - the bar which is being compared with the smallest one (see *Figure 5*).

Figure 4 – Bubble Sort Demonstration

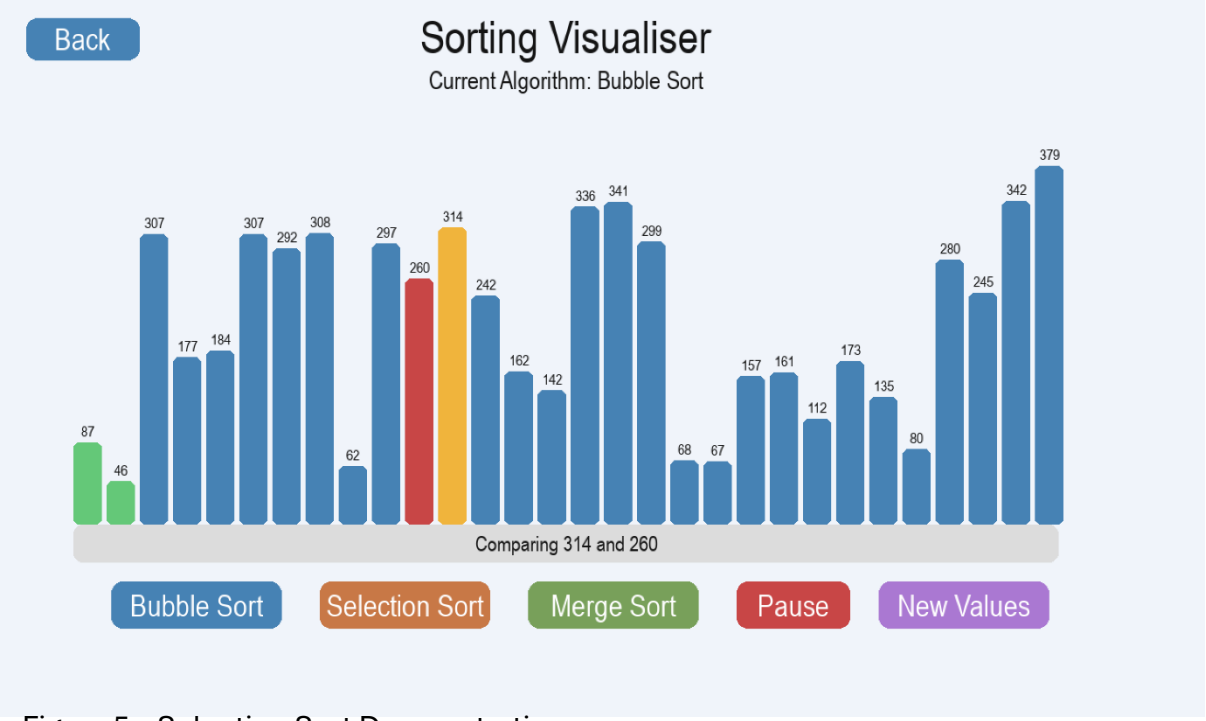


Figure 5 – Selection Sort Demonstration

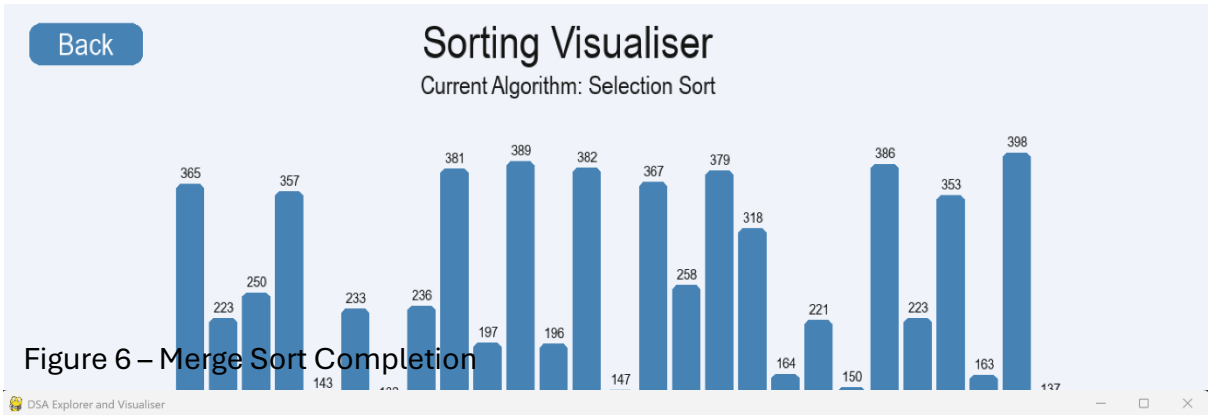
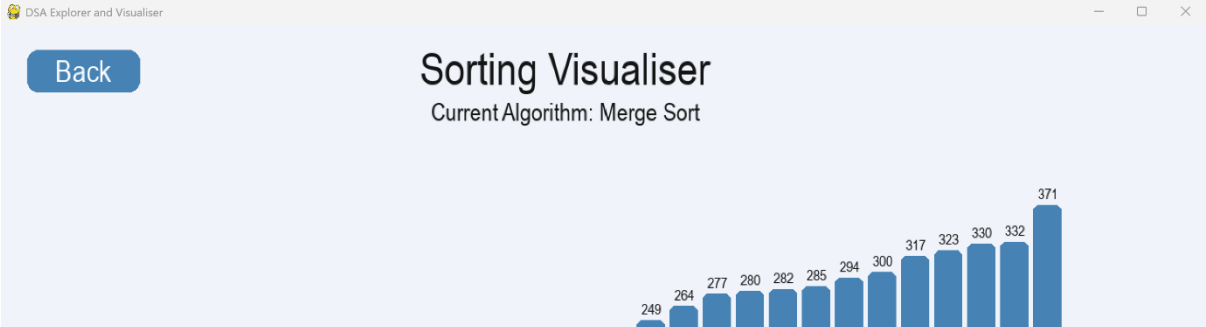


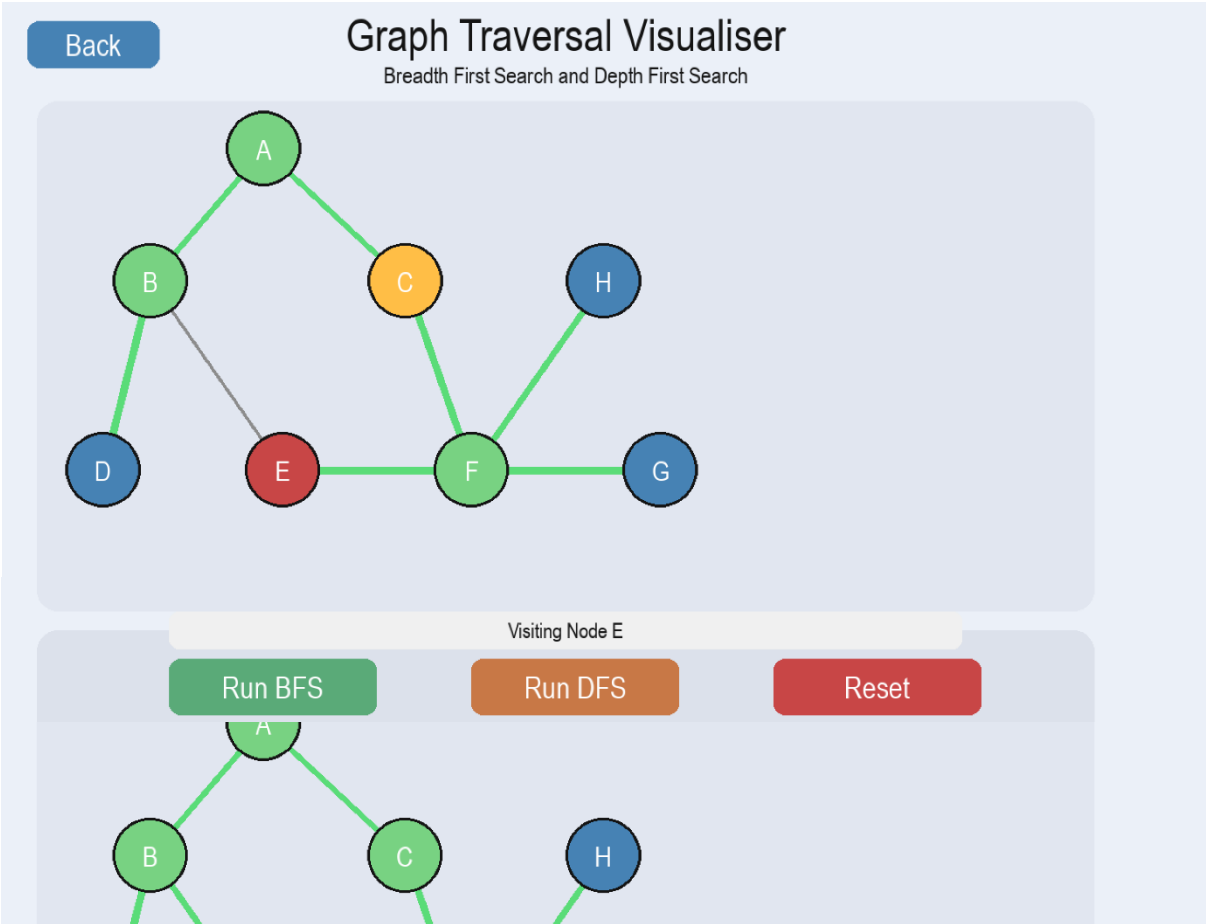
Figure 6 – Merge Sort Completion



Graph Traversal

The development of the Graph Visualiser took place within the graph_traversal.py file. Two methods of traversal implemented within this class were the implementation of breadth-first search (BFS) and depth-first search (DFS) through connections among nodes and edges. The user can select the starting point prior to execution of the BFS and DFS algorithms. Nodes change colours during traversal to show whether nodes are traversing at the moment or not. The colour red shows nodes that are being traversed while the colour green shows those nodes that have been traversed previously. This is shown in *Figures 6* and *7*.

Figure 7 – BFS Traversal



Pathfinding Module

The Puzzle Challenge module was implemented within the file known as pathfinding.py. In particular, the Dijkstra's Algorithm was designed, which determines the shortest path through a weighted graph. The graph is filled with randomly created values, which serve as the cost of movement on each cell. Moreover, some cells are created as blocked cells randomly, and it means that there is no possibility to move further. During the program running, various colours mark visited cells, blocked cells, the shortest path, and start and destination nodes. At the end of the program running, the shortest path is calculated and shown for the user (see *Figures 9* and *10*).

Figure 9 – Generate New Board

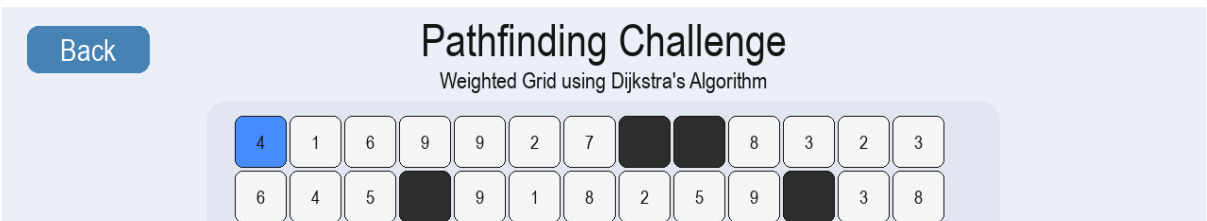
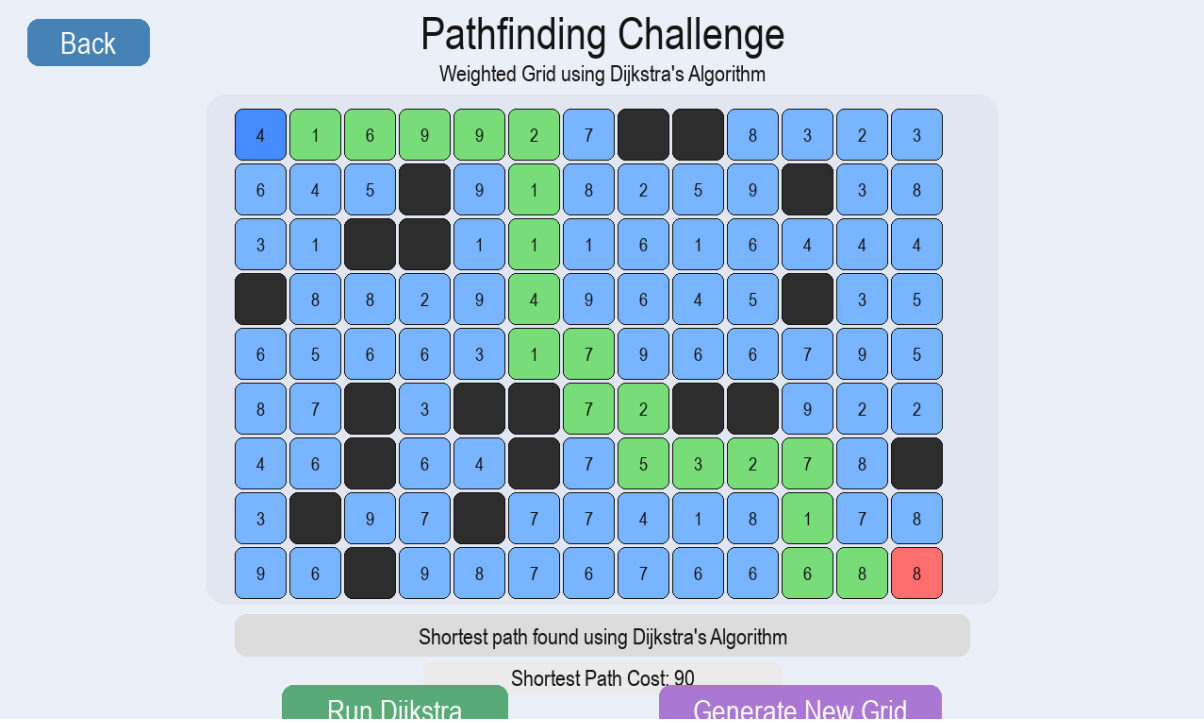


Figure 10 – Generate New Board



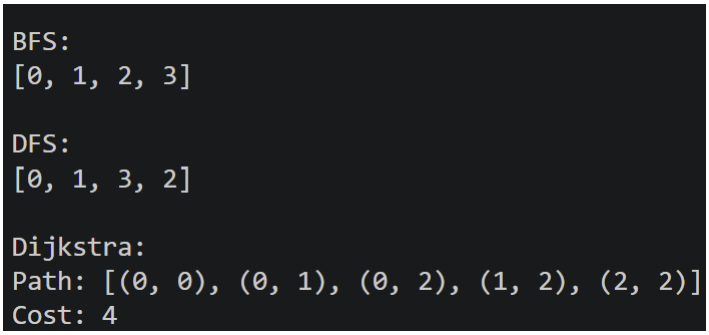
In the algorithms.py file, implementations of the algorithms were done independently of the front-end GUI. These include the implementations of the following algorithms: Bubble sort, Selection sort, Merge sort, BFS, DFS, and Dijkstra's Algorithm. The separation of these algorithms from the visualizer components helped make the code more organized, modular and easy to test within the project. The algorithms module is

mainly concerned with the logic behind the algorithm while the visualizers deal with the graphical and interactive aspects.

Testing Process

The testing framework of the project was created in the test_algorithms.py file, which includes tests for the sorting algorithms, graph traversal algorithms, and Dijkstra's algorithm for finding the shortest path. The tests were conducted to ensure that the output generated by the algorithms was accurate and independent of the graphical user interface. In the case of the sorting algorithms, the correctness of the sorted array in ascending order was ensured. In the case of BFS and DFS, the order of node traversal was validated. In Dijkstra's algorithm, the shortest path and its associated path cost were evaluated. The testing framework could be executed using the command line, as illustrated in *Figure 11*.

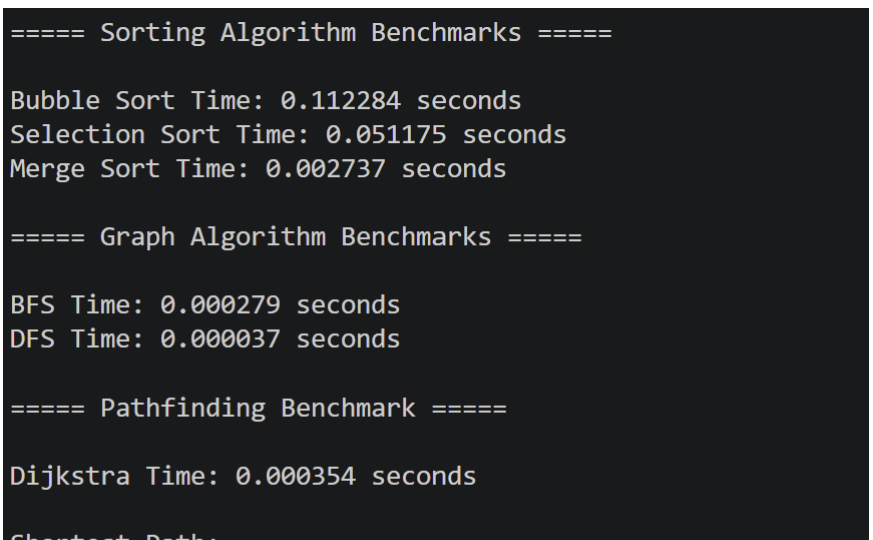
Figure 11 – Testing Results



Benchmarking

The development of benchmark testing within the benchmarks.py file allows for a comparison of the performance of the different algorithms. This file includes run-time tests of Bubble Sort, Selection Sort, Merge Sort, BFS, DFS, and Dijkstra's algorithm using the time.perf_counter() function in Python. Randomly generated datasets were used to compare the efficiency of each algorithm by timing how fast each process would complete. The outcome showed that the merge sort algorithm completed the sorting process much quicker than the bubble sort and selection sort algorithms, as it follows the divide and conquer approach. Results are shown in *Figure 12*.

Figure 12 – Benchmark Results



Evaluation

The final application successfully met the primary goal of developing a program that would provide an interactive means of learning about Data Structures and Algorithms via visualisations. The program enables the user to interact with different algorithms visually by animations, colour codes and graphic interfaces. The project structure aided in organising the work and enabled the independent development of all the modules. While the interface worked well overall, especially within the Sorting Visualiser and Puzzle Challenge modules, where the algorithms were animated, there were some problems related to interface layouts during the process of development. Some buttons had difficulties appearing correctly on the screen and needed many interface changes to be solved. Moreover, the BFS algorithm's path is not entirely outlined until the end of the traversal process. Apart from those problems, the project was successful in demonstrating how algorithms behave.

Future Improvements

The following are some of the potential future upgrades that can be considered for this project: integrating other sorting techniques, such as Quick Sort or Heap Sort; incorporating further graph traversing/pathfinding methods; refining the user interface design; enabling users to generate their own graphs or weighted grids; introducing other features in the user interface, including adjustable animation speed, sound effects, dark mode, and enhanced balancing visualizations for trees; and incorporating further enhancements in the graph traversing module that may entail visualization of the traversed paths using BFS/DFS methods.

Conclusion

The DSA Explorer and Visualiser application is thus an effective tool that shows various Data Structures and Algorithms by means of graphical visualization and user interaction using Python and Pygame. It is an effective blend of algorithm design, animations, modular programming, testing, and benchmarking within one application.

The end result of this project is the creation of an application which enables the user to gain a clearer insight into the internal working of algorithms by visualizing sorting, traversal, and path finding algorithms.